

Trabajo Práctico Final

Parte B

I-402 - Principios de la Robótica Autónoma

Prof. Ignacio Mas, Tadeo Casiraghi y Bautista Chasco

En esta segunda etapa del trabajo práctico final (Parte B), los alumnos deberán centrarse en el desarrollo del módulo de **Movimiento Automático y Navegación Autónoma**. El objetivo primordial es lograr que la plataforma robótica sea capaz de desplazarse de manera inteligente y segura entre puntos arbitrarios de un entorno previamente mapeado, utilizando como pilar la información y las representaciones espaciales consolidadas durante la Parte A.

Dependiendo del resultado obtenido en la fase de SLAM, los grupos trabajarán sobre un enfoque de navegación basado puramente en mapas de ocupación (Grilla) o bien en un sistema híbrido que combine mapas de ocupación con mapas de hitos geométricos (Landmarks). Esta etapa integra de forma directa problemas complejos de la robótica clásica y probabilística, tales como la relocalización global, la planificación óptima de trayectorias en presencia de obstáculos dinámicos, el control de seguimiento cinemático y la gestión del comportamiento del robot mediante arquitecturas reactivas y máquinas de estados.

1. Navegación Autónoma y Control

En esta sección se implementará la capacidad del robot para desplazarse de manera autónoma entre dos puntos seleccionados por el usuario. El sistema deberá resolver concurrentemente la estimación de la pose dentro del mapa estático conocido, la planificación de un camino factible libre de colisiones y la ejecución de un controlador cinemático robusto para el seguimiento de la trayectoria calculada.

1.1. Localización

De acuerdo con la arquitectura seleccionada y el mapa resultante de la Parte A, los alumnos deberán configurar su solución para la localización bajo uno de los siguientes tres sistemas de navegación expuestos en el diagrama de flujo general:

1.1.1. Sistema 1: Navegación Basada en Grilla Pura

Sin importar la opción elegida para la Parte A del trabajo final, en todos los casos se obtiene una grilla de ocupación del entorno. Con este mapa, podrán localizarse utilizando métodos de concordancia de mapas.

1.1.2. Sistema 2: Navegación Basada en Grilla + Landmarks de LIDAR

Si seleccionaron el método de landmarks de LIDAR, podrán continuar aplicando ese sistema en la Parte B. En este caso, se tendrán que localizar utilizando los hitos ya conocidos extraídos de las mediciones del LIDAR.

1.1.3. Sistema 3: Navegación Basada en Grilla + Landmarks de CÁMARA

Si seleccionaron el método de landmarks de CÁMARA, tendrán que adaptar su trabajo para poder simularlo en Gazebo. Dado que el entorno estándar de simulación en Gazebo no cuenta de forma nativa con marcadores visuales o hitos identificables idénticos a los del entorno real (RosBag), los alumnos deberán diseñar e implementar un nodo de ROS 2 que actúe como un sensor virtual de landmarks.

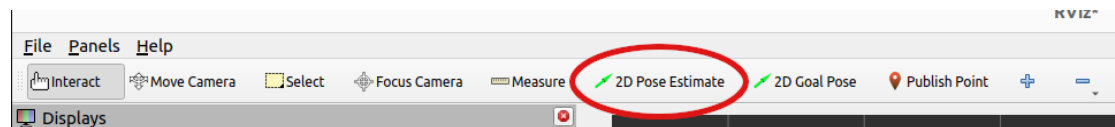
Lineamientos críticos para el nodo de Landmarks virtuales:

- **Emulación criteriosa de densidad:** Deberán poblar el mundo virtual con una cantidad y distribución de landmarks que guarde estricta coherencia con la densidad observada en las capturas del mundo real de la Parte A, garantizando un desafío matemático equivalente.
- **Lógica de oclusión (Línea de Visión):** El nodo sensor virtual debe calcular activamente la visibilidad geométrica entre el robot y los landmarks. Si un obstáculo estático o dinámico se interpone físicamente en la trayectoria directa entre el sensor del robot y el hito, el landmark debe ser catalogado como ocluido y su lectura no deberá ser publicada ni integrada en ese instante de tiempo.

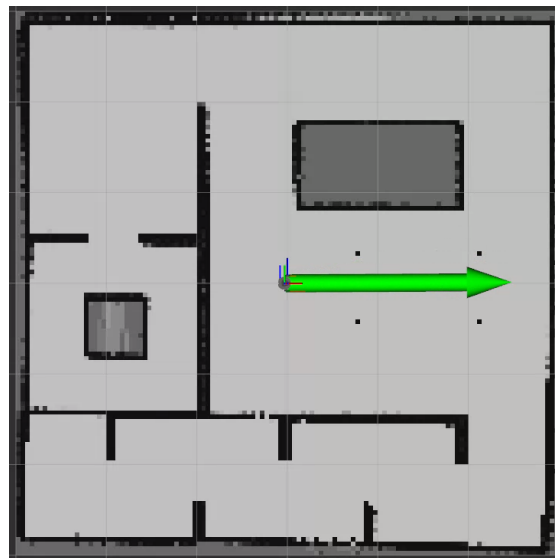
Con estos landmarks ficticios tendrán que repetir el proceso de mapeo para tener un mapa del entorno simulado.

1.2. Localización Inicial en el mapa

Al comenzar la simulación, el robot no sabrá donde está ubicado en el mapa. En la interfaz de RVIZ2 está la herramienta de "2D Pose Estimate". Esta permite fijar una posición estimada de inicio para el robot.



Al hacer click en ella y luego en la posición esperada en el mapa podremos fijar una estimación de la posición del robot.

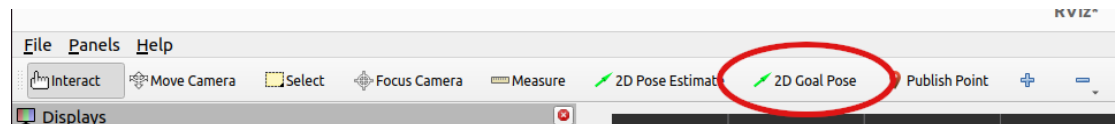


Al hacer esto se publicará automáticamente al tópico "*initialpose*" de tipo "*PoseWithCovarianceStamped*" [geometry msgs]. Deberán tomar esta primera aproximación y localizarse alrededor de esta.

1.3. Objetivo en el mapa

De manera similar, tendrán que luego definir la posición final deseada. En la interfaz de RVIZ2 está la herramienta de "2D Goal Pose". Esta permite fijar una posición final para el robot.

Al utilizarla se publicará automáticamente al tópico "*goal_pose*" de tipo "*PoseStamped*" [geometry msgs]. Deberán utilizar esta pose final para planear el recorrido.



1.4. Localización en el mapa

Una vez que se fija la posición inicial, cada vez que se la fije nuevamente y mientras se mueve el robot deberá poder localizarse en el mapa utilizando los datos de odometría, el sensor lidar, y el mapa previamente generado. Se debe trabajar con filtros probabilísticos (por ejemplo, filtro de partículas o EKF) para manejar la incertidumbre.

1.5. Planificación de ruta

Se deberá planificar el recorrido entre el inicio y el punto objetivo dentro del mapa generado, evitando chocar con paredes u obstáculos ya mapeados. El recorrido deberá ser seguro, sin pasar demasiado cerca de objetos de ser posible.

1.6. Seguimiento de trayectoria (Path Following)

Se deberá implementar un algoritmo que permita que el robot siga la trayectoria planteada y ejecutarlo de manera suave y correcta.

1.7. Ángulo final

El robot deberá no solo llegar a la posición final, sino también al ángulo final.

1.8. Repeticiones

En caso de que el robot haya llegado a la posición final, se deberá poder especificar una nueva posición final. El robot deberá nuevamente planear un recorrido y ejecutarlo. Esto también aplica si durante su recorrido se especifica otra pose objetivo. En ese caso el robot deberá re-planear y ejecutar el nuevo plan.

1.9. Obstáculos

En caso que el robot se encuentre con obstáculos en su recorrido que no estaban mapeados previamente, deberá ejecutar una política de evasión de obstáculo para poder completar el objetivo.

1.10. Entornos de prueba

El sistema deberá funcionar bien para el entorno estándar y con obstáculos simple:

- `ros2 launch turtlebot3_custom_simulation custom_casa.launch.py`
- `ros2 launch turtlebot3_custom_simulation custom_casa_obs.launch.py`

De manera opcional y a modo de desafío, pueden probar si sus algoritmos pueden recorrer todo el entorno en el mundo con más obstáculos y rutas cerradas:

- `ros2 launch turtlebot3_custom_simulation custom_casa_obs2.launch.py`

1.11. Máquina de Estados

Para controlar todo lo que el robot hace en cada caso, deberán programar una máquina de estados. Esta deberá contemplar todos los casos y posibilidades de cambio de estado.

1.12. Evaluación

Se evaluará su trabajo en base a:

- Eficiencia de localización inicial del robot
- Velocidad y eficiencia de generación del recorrido del robot
- Manejo de incertidumbre durante el recorrido
- "Seguridad" del recorrido del robot. (Básicamente no choquen plis)
- Capacidad de llegar al objetivo
- Capacidad de manejar pequeños obstáculos
- Opcional: Capacidad de manejar obstáculos complejos